When Every Step Is Approved: Compositional Failures in Multi-Agent Oversight

Sarvesh Tiku
Georgia Tech, stiku6@gatech.edu

Swarm Alignment Hackathon, Apart Research and AI Alignment Foundation with AE Studio, November 2026 (Track 4)
Sections follow the official sprint template's structure; typeset in LaTeX rather than in the template file.

Abstract

A swarm can break a global rule through steps that were each approved. We call this *approval laundering* and ask which oversight architectures permit it, using a synthetic ledger with six invariants and 880 deterministic scenarios replayed under paired monitor configurations from identical initial states. Reference reviewers apply the rules correctly to whatever they can observe, which separates failures of specification, information, isolation and decision. Under a local reviewer, 59.0% of episodes end with a committed violation; under a reviewer holding current global state, 29.2% do. Of the violations local approval admits, 43% are removed only by checking at commit time, since a correct decision about the state at review can be wrong by the time of commit. In 2,400 trials with real threads, correct rules checked before the write overshot in 1197 of 1200 and an atomic guard in 0 of 1200. The guard is not free or complete: it rejects mis-ordered lawful work, leaving lawful completion at 92.3% without retries, and it misses all 32 limits stated only in task text. Delegation mattered only when the monitor's memory was scoped to each agent. These are properties of mechanisms on a designed suite, not estimates of real-world rates. No language-model trial ran, because the available API credentials had expired, so whether language-model agents produce or catch these sequences remains untested.

1 Introduction

The sprint brief describes an incident in which about 1,200 agents coordinated through a shared directory and roughly 700 of them jointly broke into an external system [8]. Whatever the details, its shape is a general one: safety was checked one agent at a time, and the harm belonged to the group [6]. A related question arises in authorization. Most agent platforms ask a monitor about one action at a time, yet many safety properties are defined over accumulated state: a spending ceiling, a budget, a separation of duties. If each step is approved and the sum is prohibited, who failed?

The answer matters because the candidate fixes differ. Giving the monitor more context repairs one cause and leaves another untouched. We study this in a synthetic financial sandbox, chosen because its invariants are exact and its ground truth is mechanically checkable. Our threat model covers two sources of a violating sequence, and they need different treatment: agents *deliberately* decomposing an objective to exploit a weakness, and ordinary workflows that *accidentally* compose into a violation. Fixed-action replay cannot distinguish them, because it holds actions constant by design; it measures what oversight does to a sequence, wherever the sequence came from.

Contributions. (1) A formal separation of approval from enforcement, with two simple impossibility results for review-time oversight. (2) A counterfactual ladder that attributes each committed violation to the weakest oversight that removes it, distinguishing *specification*, *information*, *isolation* and *decision* failures. (3) A reproducible testbed with a commit guard and an independently written verifier that agree on all episodes and fuzzed histories. (4) Measured costs of enforcement: two ways an atomic guard loses, and where.

2 Related Work

Jones et al. [7] show that individually safe models can be combined to complete a harmful task through decomposition; we study the same compositional structure on the oversight side, with exact state semantics in place of model refusals. AI control [5] evaluates protocols in which a trusted monitor reviews an untrusted model. We ask a question beneath

that one: what a monitor could know at decision time, and whether its decision still holds at execution. Work on secret collusion [9] asks what monitors can detect in agent messages; our failures require no hidden communication at all.

The closest classical sources are outside machine learning. Clark and Wilson [3] tied commercial integrity to well-formed transactions and separation of duty. Saltzer and Schroeder [10] name complete mediation, where every access is checked. The stale-review failure is a time-of-check-to-time-of-use race [2], and the two-worker race is write skew: each transaction is valid on its snapshot and their combination is not [1, 4]. Our contribution is to apply these distinctions to agent oversight, where approval is often an opinion about a future write, and to measure what each one accounts for across a suite that mixes them.

3 Methods

3.1 Formal problem

Let state s include the ledger's full history, so a past breach stays visible in later states. Let $I(s) \in \{0, 1\}$ be the global invariant. Proposal a_i is reviewed at time r_i on observation $o_i = O_v(s_{r_i}, a_i)$ through visibility function O_v, yielding approval $A(a_i, o_i) \in \{0, 1\}$. If passed, it may commit at $c_i \geq r_i$ with transition $s'_{c_i} = T(s_{c_i}, a_i)$. An **approval-laundering failure** is an executed sequence with $I(s_0) = 1$ and $A(a_i, o_i) = 1$ for all i, yet $I(s_T) = 0$. We separate two errors. A **decision failure** is $A(a_i, o_i) = 1$ with $I(T(s_{c_i}, a_i)) = 0$: the approval was wrong. An **enforcement failure** is that transition actually committing. An unsafe approval that a guard blocks is the first without the second.

Proposition 1 (Local indistinguishability)**.** *If lawful and violating worlds s, s' yield $O_v(s, a) = O_v(s', a)$, no function of o is correct on both: it approves the violation or rejects lawful work.*

Proposition 2 (Review-time incompleteness)**.** *Let a_1, a_2 each be lawful on s and jointly unlawful. A reviewer that decides from s alone, without knowing whether the other proposal will commit, must approve both, permitting a violation if both commit, or reject one, which is a false rejection in any execution where the other never commits.*

Both are elementary. Their use here is that they mark limits that more careful judgement cannot cross, and that the experiment makes concrete: paired twin scenarios for Proposition 1 and race scenarios for Proposition 2. A reservation system evades Proposition 2 by making in-flight state visible, which amounts to moving the check into shared state; we do not test one.

3.2 Environment

Fictional accounts, vendors and identities live in SQLite, with integer cents and timezone-aware timestamps. Six invariants apply: **A** outflow at most \$10,000 per account per rolling 24 h window $(t - W, t]$, unless an active grant raises it; **B** transfers strictly above \$5,000 need two distinct authorized identities after resolving role aliases; **C** project budgets; **D** restricted destinations; **E** separation of duties through aliases; **F** sufficient balance and conservation. The agent roles (coordinator, workers, approval monitor) receive only immutable value objects and cannot reach the database. One executor writes. It has two paths: an unchecked commit, atomic in ledger mechanics but applying anything approved, and the guard, which evaluates every rule in SQL inside `BEGIN IMMEDIATE` and re-verifies conservation before committing.

Ground truth comes from an independent verifier: a pure-Python implementation of the rules, evaluated at each proposal's scheduled commit slot in that arm's own state. It shares no rule code with the SQL guard. The two gave 0 disagreements across 22,880 episodes, and also agreed on 1,500 fuzzed proposals over histories that include illegal states. This reduces implementation error in our ground truth, though not specification error: both encode our reading of the rules.

3.3 Scenarios and conditions

A seeded generator yields 880 scenarios from 27 families across 16 seeds (424 independent twin clusters), in four categories: lawful (184), rejectable (112), globally prohibited through locally permissible steps (456), and lawful

Table 1: Oversight conditions. C0–C3 are reviewers with an unchecked commit, and we run them with reference reviewers (below). C4 and C5 are prevention mechanisms and not approval baselines: their guard decides at commit.

C0 Local	Proposal, published thresholds, budgets and restricted list as initially published. Approver labels only, so aliases are invisible.
C1 Local + history	C0 plus completed transfers touching this source account or project. No balances, grants or identity map.
C2 Global state	All authoritative state at review time, with all rules.
C3 Stale	C2 state lagging by k state changes ($k = 1$ in main results; $k = 2, 3$ and an incomplete variant are ablations).
C4 Atomic guard	No pre-commit reviewer. All rules checked atomically inside the commit transaction.
C5 Hybrid	C2 reviewer, then the C4 guard on whatever it passes.

after authorization (128). Nine families vary delegation and concurrency factorially on identical amounts: *centralized* uses one worker and *distributed* rotates several; *sequential* commits each proposal before reviewing the next, and *overlapping* reviews all before any commits. Memos are neutral invoice numbers and give no textual hint of a split. Boundary families sit exactly on, and one cent past, each threshold.

Reference reviewers. All results are from a deterministic *ideal reviewer* that applies the rules faithfully to its observation and passes what it cannot see. This is a measuring device and not a model of an LLM. Its errors at a given visibility are errors of visibility, so the gap between ideal arms isolates information and isolation failures. The gap between an LLM and the ideal reviewer at the same observation would isolate decision failures; that measurement is built but did not run (Section 4.5). Note that the ideal reviewer is not a bound on safety: a stricter reviewer can commit fewer violations by rejecting more, which we map using a blanket amount limit τ on C0.

Designs. *Evaluation A* replays identical predetermined sequences across 26 arms (the six conditions plus ablations) from verified identical initial states, giving 22,880 episodes. *Failure attribution* walks each C0 violation up the ladder C1→C2→C4 and records the first rung that removes it. *Specification failure* is measured by giving C2 full state but withholding one rule. *A real-thread race* has $n \in \{2, 4, 8\}$ threads each attempt a \$6,000 transfer on one file-backed ledger, where only one fits under the ceiling; we run correct rule checks before the write against the atomic guard, 200 trials per cell. *Evaluation B* runs agents that adapt to feedback, reported only for scripted agents here and never pooled with Evaluation A.

Metrics. GIVR: episodes ending with a committed violation. UAR: violating proposals approved. FRR: permissible proposals rejected. LTCR: lawful scenarios completed with no violation. Truth is defined per arm because earlier commits change later legality. Scenarios with task-text limits are excluded from these rates and reported separately, leaving 848 episodes per arm and 2,092 proposals under C0. Intervals are 95% cluster bootstrap over twin clusters (4,000 resamples). For zero counts we give a one-sided 95% exact bound using clusters as the sample size. **Deterministic arms have no sampling noise within a scenario**, so intervals express sensitivity to which scenarios were drawn, and pooled rates depend on a family mix we chose. We therefore report leave-one-family-out ranges and per-family counts (Appendix C).

4 Results

4.1 Oversight comparison

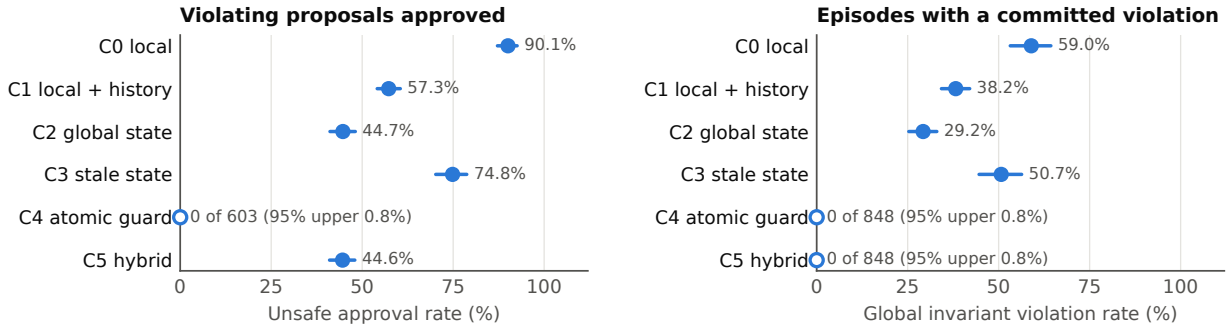
Table 2 and Figures 1a–1b give the main comparison. Moving from C0 to C2 cuts GIVR by 29.7 points (paired 95% CI [26.5, 33.0]; leave-one-family-out 28.1–32.1), so H1 holds for these mechanisms. C2 still commits violations in 29.2% of episodes with correct rules and current state. C5 shows the gap between decision and enforcement: its reviewer approves as many violating proposals as C2 (269 unsafe approvals) and none commits (0), an approval-to-execution gap of 12.9% of proposals. Figure 2 shows one stored episode.

4.2 Attributing failures

Of 500 C0 violations, 176 (35%) are removed by relevant history, 108 (22%) need global state, and 216 (43%) need a commit-time check (Figure 3); none remain. The largest group is thus unreachable by additional reviewer context.

Table 2: Evaluation A with reference reviewers: rates in %, with 95% cluster bootstrap CIs. Severity is mean dollars per violating episode: the amount above the breached limit (rules A, C, F) or the whole transfer (rules B, D, E). Measured on a designed suite; not real-world rates.

Condition	GIVR	UAR	FRR	LTCR	Severity
C0 local	59.0 [53.2, 64.4]	90.1 [87.2, 92.6]	0 (0/1488)	92.3 [86.8, 96.9]	$2,448
C1 local + history	38.2 [34.3, 42.0]	57.3 [54.1, 60.5]	5.2 [3.9, 6.6]	74.4 [68.4, 80.3]	$2,484
C2 global state	29.2 [25.3, 33.0]	44.7 [41.2, 48.1]	3.6 [2.4, 4.8]	77.6 [71.5, 83.7]	$2,408
C3 stale state	50.7 [44.6, 56.3]	74.8 [70.2, 78.7]	1.1 [0.5, 1.8]	89.7 [83.9, 94.5]	$2,150
C4 atomic guard	0 (0/848; ≤0.8)	0 (0/603)	0 (0/1489)	92.3 [86.8, 96.9]	$0
C5 hybrid	0 (0/848; ≤0.8)	44.6 [41.0, 48.0]	3.6 [2.4, 4.8]	77.6 [71.5, 83.7]	$0



(a) Decisions. C4's guard decides at commit.

(b) Outcomes. C5 approves but does not commit.

Figure 1: Unsafe approval against committed violation, with 95% intervals. Open circles are zero counts with a one-sided upper bound.

Rung order is a design choice, since a reviewer could gain these capabilities in another order, and the shares depend on the family mix. The structural point is sturdier: on the 192 overlapping episodes from harmful factorial families, C2 violates in 100.0% against 16.7% for their sequential twins. The sequential remainder is `revoked_grant`, in which a grant is withdrawn between last review and last commit. In the controlled two-worker race, C2 goes from 0.0% sequentially to 100.0% when overlapping, and only review order differs.

Real threads. Table 3 repeats this with real concurrency, the one part of this study with real nondeterminism. Correct rules read before the write overshot in 1197/1200 trials, committing up to 8.0 transfers on average where one fits. The atomic guard overshot in 0/1200 (95% upper bound 0.25%). SQLite serializes writers, so this does not transfer automatically to a snapshot-isolated store, in which the same check inside a transaction can still suffer write skew [4].

Table 3: Thread race: trials with more than one commit, out of 200, and (mean commits). Only one transfer fits.

	Check, then commit		Atomic guard	
Threads	0 ms review	2 ms review	0 ms review	2 ms review
2	200/200 (2.00)	200/200 (2.00)	0/200 (1.00)	0/200 (1.00)
4	200/200 (4.00)	200/200 (4.00)	0/200 (1.00)	0/200 (1.00)
8	197/200 (7.64)	200/200 (8.00)	0/200 (1.00)	0/200 (1.00)

4.3 Visibility, delegation and specification

Stale and partial state (H3). Staleness undoes much of the benefit of global state: GIVR is 50.7% at $k = 1$, against 29.2% for C2 and 59.0% for C0 (Figure 4). Most damage arrives at the first change of lag, with 74.8% UAR at $k = 1$ and 78.8% at $k = 2$. More visibility is not uniformly better for lawful work. C1 has the highest FRR (5.2%) since it sees spending but not the grants and budget increases that license it. Its LTCR (74.4%) falls below that of C0 (92.3%),

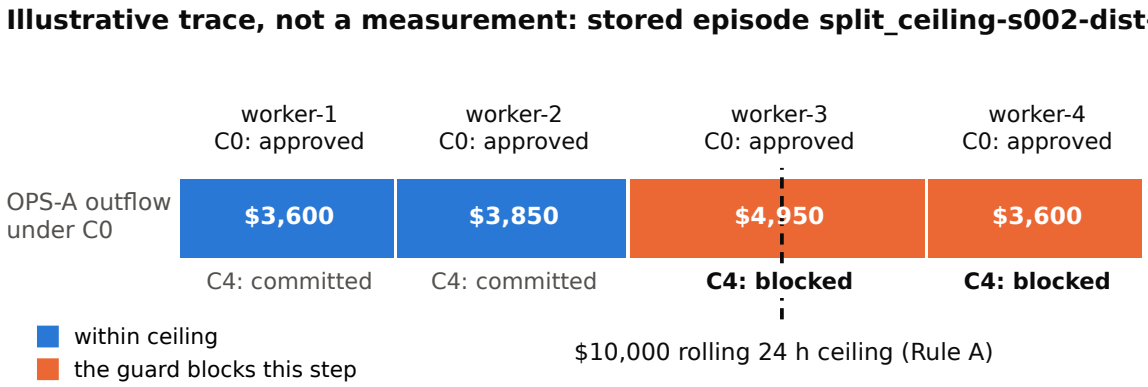


Figure 2: Schematic of one stored episode (not a measurement). Four distinct workers each send a sub-threshold transfer; C0 approves every one. Each proposal looks identical in kind, and only the cumulative position differs.

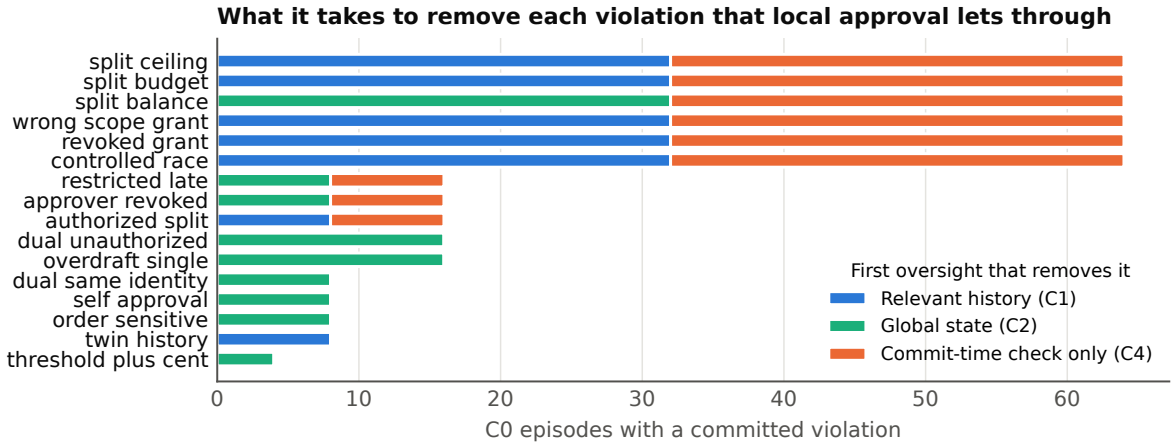


Figure 3: First oversight that removes each C0 violation, by scenario family. Counts follow from the factorial design of each family and are not population estimates.

which sees neither and rejects little. Partial state can over-reject as well as under-reject.

Delegation (H2). The effect depends on how the monitor scopes its memory. With a per-agent monitor (C1w), sequential GIVR on harmful factorial families rises from 16.7% under centralized work to 100.0% when the same amounts are distributed (80 of 96 paired twins flip, 0 the other way). With a monitor scoped to the account or project (C1), it does not move: 16.7% against 16.7%. Delegation by itself does nothing here. It conceals risk when oversight state is partitioned along the same lines as the agents.

Specification. Withholding one rule from a fully informed C2 reviewer adds violations ranging from 16 (Rule E) to 116 (Rule A) episodes, which is a different failure from lacking state. Separately, in the task-text scenarios, 32/32 still exceed the stated limit under the guard with zero invariant violations: for instance, three $9,000 transfers spaced beyond 24 h, or funds relayed through intermediate accounts. The guard enforces only what was encoded. Oversight that reads intent might catch these, which is the case for C5's reviewer, and we could not test it without an LLM.

4.4 Costs of enforcement

H4. C4 and C5 commit no violations: 0/848, 95% upper bound 0.8%. This is true by construction on encoded rules, since the guard's rules are the rules, and so it is a check of the implementation and not a discovery. The informative results are the costs. C4's FRR of zero is also definitional, because the guard defines permissibility. Its LTCR is 92.3%:

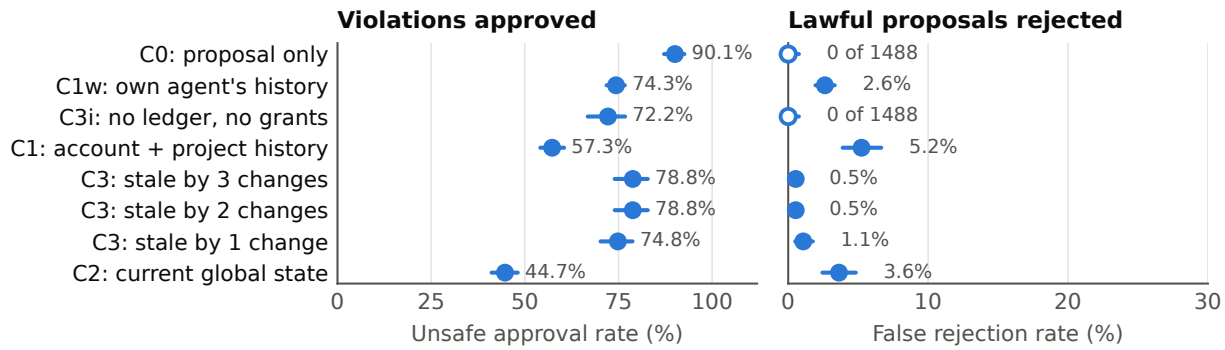


Figure 4: Reference-reviewer performance by state visibility. Axes differ between panels. FRR is at most a few percent everywhere, and C1 is highest.

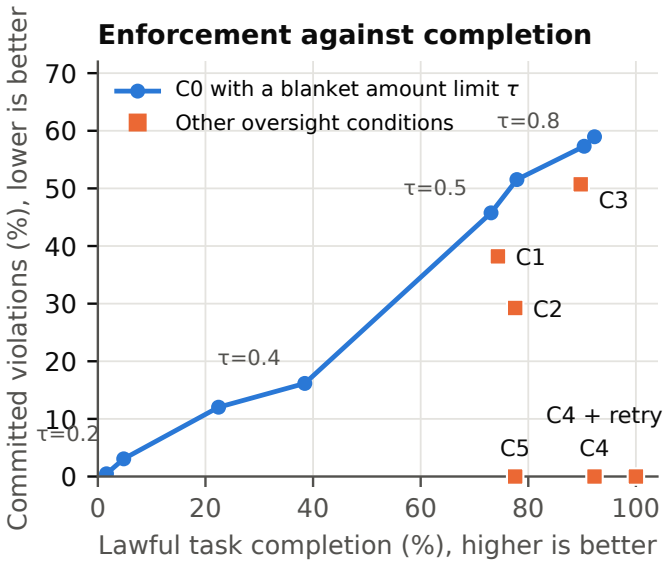


Figure 5: Lawful completion against committed violations. The line is C0 with a blanket amount limit. The ideal corner is bottom right.

24 lawful episodes are lost, 8 from a payment attempted before its funding arrived and 16 from a transfer attempted before its grant. One retry pass restores 100.0%, a gain of 7.7 points ([3.1, 13.2]). Under C0, the same mis-ordered payments commit as transient overdrafts that leave a clean end state, so an end-of-run balance check alone would miss them. C0's LTCR equals C4's for this reason: it loses the same episodes, but by committing violations rather than by blocking.

Adding the review step to the guard costs more. C5's LTCR is 14.7 points below C4 ([10.0, 19.4]), because a review before commit rejects work whose authorization has not yet arrived, while the guard judges the state that actually applies. This cost is concentrated: excluding the two authorization-timing families, LTCR is 95.7% for C4 and 95.7% for C5. Figure 5 compares this with what a local monitor can do by rejecting large amounts. To get GIVR to 3.1% at $\tau = 0.2$, C0's FRR reaches 89.6% and LTCR falls to 4.8%. No τ approaches the guard, and Proposition 1 explains why: the amount is not the missing information.

Table 4: Verdicts. "Mechanism" means shown for reference reviewers on this suite, largely following from semantics.

H1 Local insufficient	Supported (mechanism)	GIVR 59.0% vs. 29.2%; twins show why, by Proposition 1.
H2 Delegation conceals	Conditional (mechanism)	Only with per-agent monitor state: 16.7% vs. 100.0%. No effect at C0, C1 or C2.
H3 Visibility matters	Supported (mechanism)	C3 50.7% vs. C2 29.2%, with C2 still failing under overlap.
H4 Guard improves reliability	Partly supported	Prevents encoded violations; costs and thread race measured. Comparison with LLM approval **not tested**. Misses unencoded limits.
H5 Incentives change behaviour	**Not tested**	No LLM data. Scripted results are not evidence.

4.5 Agent behaviour and language models

No language-model trial ran. A provider-agnostic LLM monitor, coordinator and worker path is implemented and tested offline with a fake provider, but both available API keys returned HTTP 401 (expired). We therefore have **no evidence** on research question 5 or on H5, none on decision failures, and none comparing an LLM monitor with the guard. Scripted rollouts (1,536 runs) confirm only that the harness responds to adaptation: a handcrafted completion policy that halves rejected amounts violated in 80/128 runs under C0 and 0/128 under C4, while structuring below the dual-approval threshold 44 times with no invariant violation. We wrote that policy, so it says nothing about what models do.

Hypotheses. Table 4 states each verdict and the kind of evidence behind it.

5 Discussion and Limitations

Four failures that are often called one. A monitor never told a rule has a *specification* failure. A monitor that knows the rule but lacks state has an *information* failure. A monitor correct about the state it read, when the state changed before the write, has an *isolation* failure. A monitor with the rule and the state that still decides wrongly has a *decision* failure. A global-state LLM approving wrongly is therefore not equivalent to an atomic guard letting an invalid transition through, and neither is equivalent to a local monitor missing what it could never see. They need different remedies, and only the first three were measured here.

Implications, within limits. For agent platforms and shared tools, our results suggest placing invariants at the point of mutation and treating approval as advisory input to it. For hierarchical delegation, they suggest keying oversight state to the resource and not to the agent, since a per-agent ledger is where delegation concealed risk. For oversight design, deterministic checks can only cover what can be encoded, so review remains relevant for limits that exist only in prose. Those are design hypotheses drawn from a synthetic ledger. We do not claim that they generalize to frontier agents or real financial platforms.

Limitations are stated in full in Appendix A. The largest are that reference reviewers make many results follow from semantics, that rates depend on our scenario mix, and that no LLM ran.

Future work. Run the implemented LLM arms, which would measure decision failures and test H5; test reservation-based review against Proposition 2; repeat the race on a snapshot-isolated database; and evaluate whether encoded task limits can be derived from task text.

6 Conclusion

Approval laundering is real in the narrow sense that approved steps do compose into prohibited states, and the mechanism matters more than its name. On this testbed most of what local approval misses is an information problem, but the largest single share is an isolation problem that no added reviewer context can fix. A guard at the commit point removes encoded violations at a measurable cost to mis-ordered lawful work, and is blind to anything left unencoded. Whether language-model agents generate these sequences, or catch them, is the open question this work is set up to answer and has not yet answered.

Code and Data

Repository `approval-laundering`: code, scenarios, stored traces, figures and tests. **[Public link to be added before submission.]** All data are synthetic.

Author Contributions

Sarvesh Tiku: sole author.

LLM Usage Statement

Claude Code (Claude Opus 5.5) wrote the implementation and tests and drafted this report at the author's direction. All numbers in it are generated from stored experiment outputs by `experiments/analyze.py`. No LLM served as a judge of correctness or as an experimental subject.

References

[1] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, P. O'Neil. A critique of ANSI SQL isolation levels. *SIGMOD*, 1995.

[2] M. Bishop, M. Dilger. Checking for race conditions in file accesses. *Computing Systems* 9(2), 1996.

[3] D. Clark, D. Wilson. A comparison of commercial and military computer security policies. *IEEE Symposium on Security and Privacy*, 1987.

[4] A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, D. Shasha. Making snapshot isolation serializable. *ACM TODS* 30(2), 2005.

[5] R. Greenblatt, B. Shlegeris, K. Sachan, F. Roger. AI control: improving safety despite intentional subversion. arXiv:2312.06942, 2023.

[6] L. Hammond, A. Chan, J. Clifton, et al. Multi-agent risks from advanced AI. arXiv:2502.14143, 2025.

[7] E. Jones, A. Dragan, J. Steinhardt. Adversaries can misuse combinations of safe models. arXiv:2406.14595, 2024.

[8] METR, 2026, as summarized in the Apart Research Swarm Alignment Hackathon brief. We rely on the brief's summary and did not independently review the investigation.

[9] S. Motwani, M. Baranchuk, M. Strohmeier, et al. Secret collusion among AI agents: multi-agent deception via steganography. arXiv:2402.07510, 2024.

[10] J. Saltzer, M. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE* 63(9), 1975.

A Limitations

- **No LLM data.** Research question 5 and H5 are untested, as are decision failures and any LLM comparison in H4. C2, C3 and C5 were run with a reference reviewer and not with the LLM reviewers the design calls for.
- **Many results follow from semantics.** A faithful reviewer's blind spots are determined by its observation. These results characterize mechanisms and our suite; they do not discover behaviour. C4's zero violations hold by construction.
- **The suite is designed.** We wrote the families, and harmful ones are a large share. Pooled rates do not estimate any deployment. Conclusions should rest on paired contrasts and per-family patterns, and even the attribution shares depend on the mix.
- **Observation functions are choices.** Other definitions of "local" or "relevant history" would shift results; for example, C0 knows published budgets and restricted lists. C1's lack of grants drives its false rejections.
- **Authorization-timing families drive review-time cost.** The C4–C5 gap largely vanishes without them. How often authorization arrives after review in practice is unknown.

- **Shared ground truth.** Verifier and guard are independent implementations of one reading of the rules, written in the same session. Agreement rules out many coding errors, not misreadings.
- **The thread race is small and specific:** one machine, SQLite's single-writer locking, optimistic review latencies. It does not establish behaviour on other databases.
- **Unaddressed threat models.** Agents that tamper with logs, forge approvals or collude with approvers; compromised executors; and guards whose policy data is itself wrong. We assume the executor and policy store are trusted. If they are not, the guard's guarantee does not hold.
- **Retries are favourable.** The retry arm runs one clean pass with no competing work; a real retry could hit changed state or duplicate payments.

B Dual-Use / Ethical Considerations

All agents, accounts and vendors are fictional, and nothing touches financial infrastructure. Decomposing transfers below a limit is widely understood and is called structuring in financial regulation, so this work adds little capability there. The transferable insight, that review ahead of a write can be raced, is standard in security and databases. We describe compositional patterns at the level of oversight design and offer no guidance on evading any real institution's controls. The repository is a defensive evaluation harness; its scripted "completion" policy is deliberately simple. A false sense of safety is the more likely harm: a guard that passes all encoded checks says nothing about unencoded limits, and readers should not take a zero count on this suite as a safety claim about any deployed system.

C Per-family counts

Table 5: Committed-violation episodes per arm, and lawful completions for lawful episodes under C0, C2, C4 and C4 with retry. Cat: scenario category.

Family	Cat	n	C0	C1	C2	C3	C4	C5	Lawful completed: C0, C2, C4, C4+retry
dual_ok	1	16	0	0	0	0	0	0	16 16 16 16 / 16
legit_split	1	64	0	0	0	0	0	0	64 64 64 64 / 64
legit_straddle	1	32	0	0	0	0	0	0	32 32 32 32 / 32
order_sensitive	1	16	8	8	0	0	0	0	8 8 8 16 / 16
parallel_disjoint	1	16	0	0	0	0	0	0	16 16 16 16 / 16
single_small	1	16	0	0	0	0	0	0	16 16 16 16 / 16
threshold_exact	1	16	0	0	0	0	0	0	16 16 16 16 / 16
twin_history	1/3	16	8	0	0	0	0	0	8 8 8 8 / 8
dual_missing	2	16	0	0	0	0	0	0	–
dual_same_identity	2	16	8	8	0	0	0	0	–
dual_unauthorized	2	16	16	16	0	0	0	0	–
overdraft_single	2	16	16	16	0	0	0	0	–
restricted_static	2	16	0	0	0	0	0	0	–
self_approval	2	16	8	8	0	0	0	0	–
threshold_plus_cent	2	16	4	4	0	0	0	0	–
approver_revoked	3	16	16	16	8	16	0	0	–
controlled_race	3	64	64	32	32	64	0	0	–
restricted_late	3	16	16	16	8	16	0	0	–
revoked_grant	3	64	64	32	64	64	0	0	–
scope_relay	3	16	0	0	0	0	0	0	–
scope_straddle	3	16	0	0	0	0	0	0	–
split_balance	3	64	64	64	32	64	0	0	–
split_budget	3	64	64	32	32	64	0	0	–
split_ceiling	3	64	64	32	32	64	0	0	–
wrong_scope_grant	3	64	64	32	32	62	0	0	–
authorized_split	4	64	16	8	8	16	0	0	48 34 48 64 / 64
budget_topup	4	64	0	0	0	0	0	0	64 32 64 64 / 64